class 013

Rongyu Chen(A17708249)

2026-05-18

Import countData and colData

```
# Complete the missing code
counts <- read.csv("airway_scaledcounts.csv", row.names=1)
metadata <-  read.csv("airway_metadata.csv")
```

Let's have a wee peak at these objects:

```
head(counts)
```

```
##                 SRR1039508 SRR1039509 SRR1039512 SRR1039513 SRR1039516
## ENSG00000000003        723        486        904        445       1170
## ENSG00000000005          0          0          0          0          0
## ENSG00000000419        467        523        616        371        582
## ENSG00000000457        347        258        364        237        318
## ENSG00000000460         96         81         73         66        118
## ENSG00000000938          0          0          1          0          2
##                 SRR1039517 SRR1039520 SRR1039521
## ENSG00000000003       1097        806        604
## ENSG00000000005          0          0          0
## ENSG00000000419        781        417        509
## ENSG00000000457        447        330        324
## ENSG00000000460         94        102         74
## ENSG00000000938          0          0          0
```

Q1. How many genes are in this dataset?

```
nrow(counts)
```

```
## [1] 38694
```

38,694 genes.

```
metadata
```

```
##           id     dex celltype     geo_id
## 1 SRR1039508 control   N61311 GSM1275862
## 2 SRR1039509 treated   N61311 GSM1275863
## 3 SRR1039512 control  N052611 GSM1275866
## 4 SRR1039513 treated  N052611 GSM1275867
## 5 SRR1039516 control  N080611 GSM1275870
## 6 SRR1039517 treated  N080611 GSM1275871
## 7 SRR1039520 control  N061011 GSM1275874
## 8 SRR1039521 treated  N061011 GSM1275875
```

Q2. How many 'control' cell lines do we have?

```
sum(metadata$dex == "control")
```

```
## [1] 4
```

4 'control' cell lines.

Toy differential gene expression

Lets perform some exploratory differential gene expression analysis. Note: this analysis is for demonstration only. NEVER do differential expression analysis this way!

> Q3. How would you make the above code in either approach more robust? Is there a function that could help here?

To make the code more robust, use rowMeans() instead of dividing by a fixed number so the calculation automatically adjusts to any number of samples.

```
#Step 1. Extracted all the "control" columns
control.inds <- metadata$dex == "control"
control.counts <- counts[,control.inds]
head(counts[,control.inds])
```

```
##                 SRR1039508 SRR1039512 SRR1039516 SRR1039520
## ENSG00000000003        723        904       1170        806
## ENSG00000000005          0          0          0          0
## ENSG00000000419        467        616        582        417
## ENSG00000000457        347        364        318        330
## ENSG00000000460         96         73        118        102
## ENSG00000000938          0          1          2          0
```

```
#Step 2, mean of all control columns.
control.means <- rowMeans(control.counts)
head(control.means)
```

```
## ENSG00000000003 ENSG00000000005 ENSG00000000419 ENSG00000000457 ENSG00000000460
##          900.75            0.00          520.50          339.75           97.25
## ENSG00000000938
##            0.75
```

> Q4. Follow the same procedure for the treated samples (i.e. calculate the mean per gene across drug treated samples and assign to a labeled vector called treated.mean)

For the treated samples, select columns where `dex == "treated"` and calculate their per-gene averages using `rowMeans()` to create the `treated.means` vector.

```
#Step 3. Extracted all the "treated" columns
treated.inds <- metadata$dex == "treated"
treated.counts <- counts[,treated.inds]
head(counts[,treated.inds])
```

```
##                 SRR1039509 SRR1039513 SRR1039517 SRR1039521
## ENSG00000000003        486        445       1097        604
## ENSG00000000005          0          0          0          0
## ENSG00000000419        523        371        781        509
## ENSG00000000457        258        237        447        324
## ENSG00000000460         81         66         94         74
## ENSG00000000938          0          0          0          0
```

```
#Step 4, mean of all treated columns.
treated.means <- rowMeans(treated.counts)
head(treated.means)
```

```
## ENSG00000000003 ENSG00000000005 ENSG00000000419 ENSG00000000457 ENSG00000000460
##          658.00            0.00          546.00          316.50           78.75
## ENSG00000000938
##            0.00
```

Store these together for ease of bookkeeping as `meancounts`

```
meancounts <- data.frame(control.means, treated.means)
head(meancounts)
```

```
##                 control.means treated.means
## ENSG00000000003        900.75        658.00
## ENSG00000000005          0.00          0.00
## ENSG00000000419        520.50        546.00
## ENSG00000000457        339.75        316.50
## ENSG00000000460         97.25         78.75
## ENSG00000000938          0.75          0.00
```

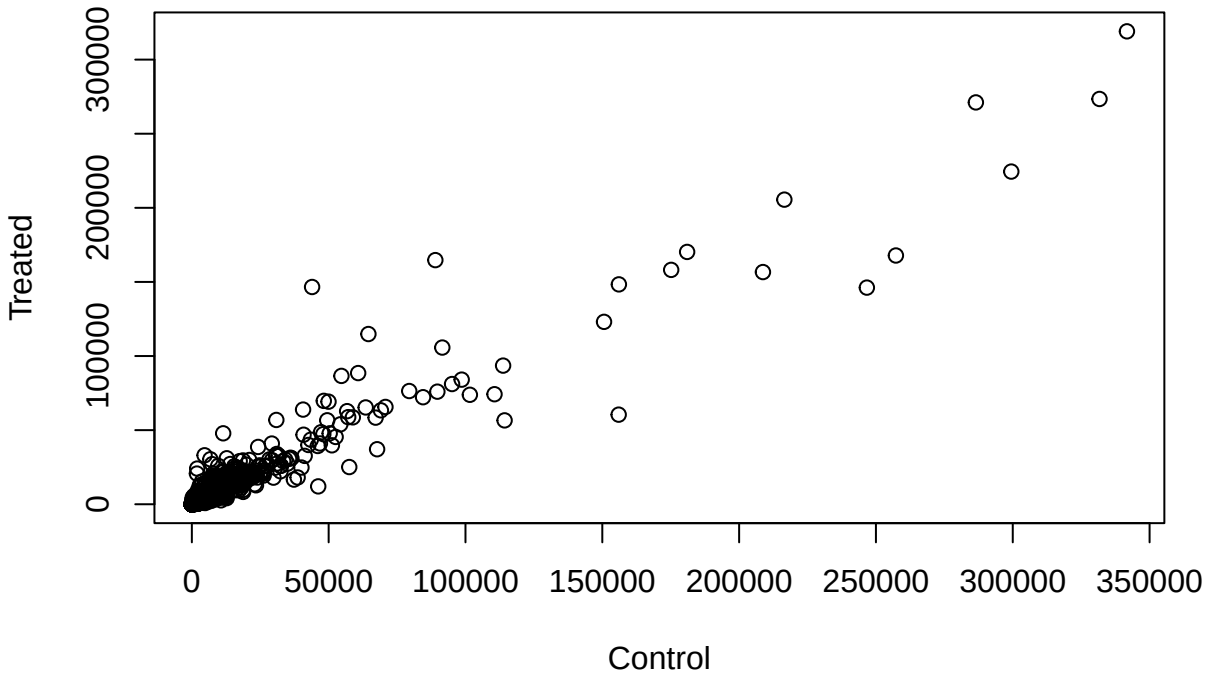
```
colSums(meancounts)
```

```
## control.means treated.means
##      23005324      22196524
```

> Q5 (a). Create a scatter plot showing the mean of the treated samples against the mean of the control samples. Your plot should look something like the following.

Make a plot of control vs treated mean values

```
plot(meancounts, xlab="Control", ylab="Treated")
```

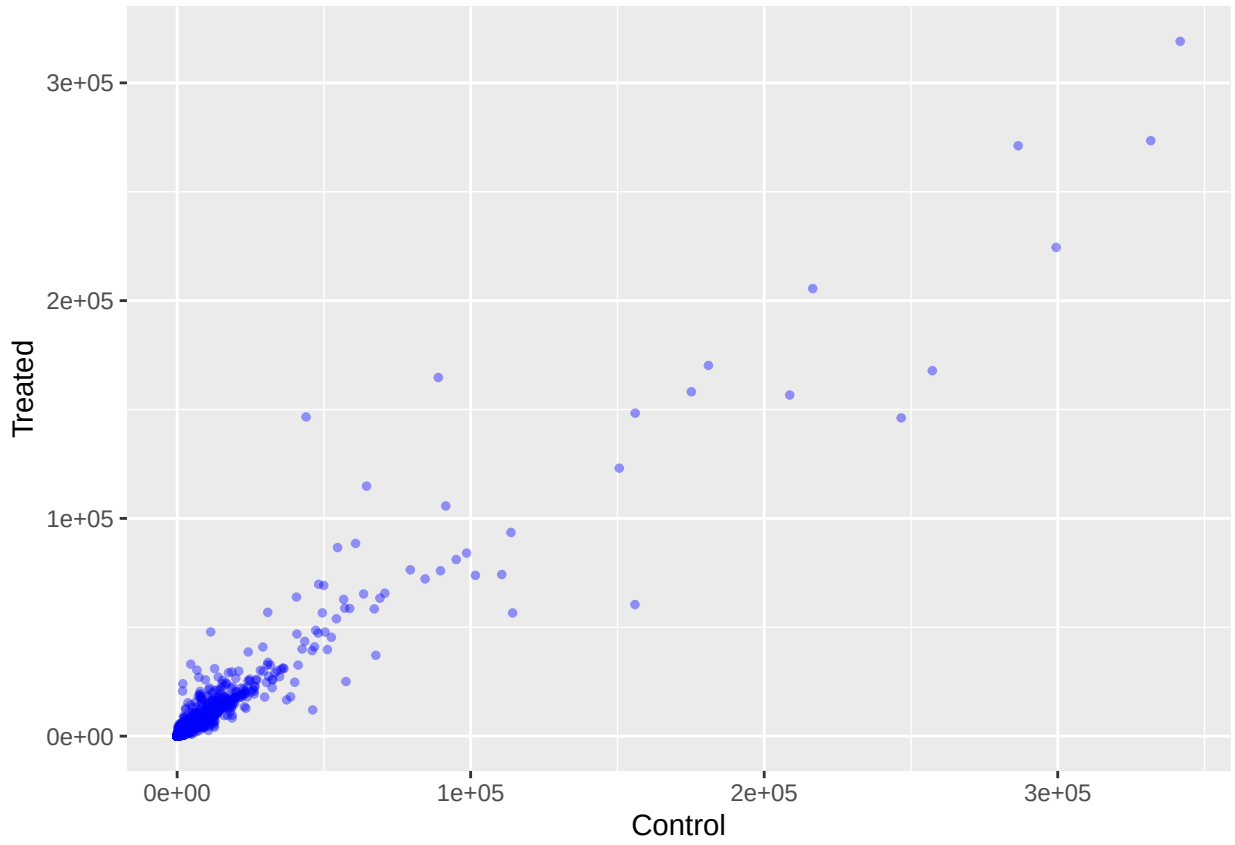


Q5 (b).You could also use the ggplot2 package to make this figure producing the plot below. What geom_?() function would you use for this plot?

use the geom_point() function.

```
library(ggplot2)

ggplot(meancounts, aes(x = control.means, y = treated.means)) +
  geom_point(color = "blue", alpha = 0.4, size = 1) +
  labs(x = "Control", y = "Treated")
```



Make this a log log plot.

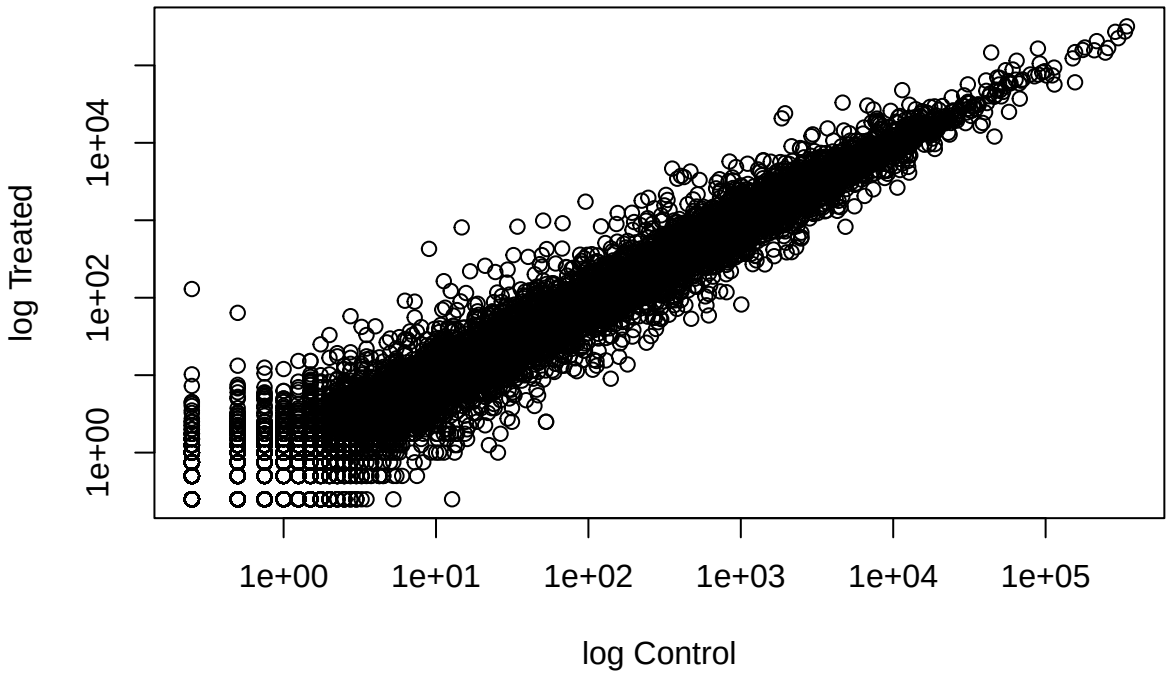
> Q6. Try plotting both axes on a log scale. What is the argument to plot() that allows you to do this?

Use the `log = "xy"` argument in `plot()`.

```
plot(meancounts, log = "xy", xlab = "log Control", ylab = "log Treated")
```

```
## Warning in xy.coords(x, y, xlabel, ylabel, log): 15032 x values <= 0 omitted
## from logarithmic plot
```

```
## Warning in xy.coords(x, y, xlabel, ylabel, log): 15281 y values <= 0 omitted
## from logarithmic plot
```



We often talk metrics like "log2 fold-change"

```
# treated/control
# 0=no change, positive=up regulated, negative=down regulated
log2(10/10)
```

```
## [1] 0
```

```
log2(10/20) # half the amount
```

```
## [1] -1
```

```
log2(20/20) # double the amount
```

```
## [1] 0
```

Let's calculate the log2 fold change for our treated over control mean counts.

```
meancounts$log2fc <- log2(meancounts$treated.means/meancounts$control.means)
head(meancounts)
```

```
##                 control.means treated.means      log2fc
## ENSG00000000003        900.75        658.00 -0.45303916
## ENSG00000000005          0.00          0.00         NaN
## ENSG00000000419        520.50        546.00  0.06900279
## ENSG00000000457        339.75        316.50 -0.10226805
## ENSG00000000460         97.25         78.75 -0.30441833
## ENSG00000000938          0.75          0.00        -Inf
```

```
zero.vals <- which(meancounts[,1:2]==0, arr.ind=TRUE)

to.rm <- unique(zero.vals[,1])
mycounts <- meancounts[-to.rm,]
head(mycounts)
```

```
##                 control.means treated.means      log2fc
## ENSG00000000003        900.75        658.00 -0.45303916
## ENSG00000000419        520.50        546.00  0.06900279
## ENSG00000000457        339.75        316.50 -0.10226805
## ENSG00000000460         97.25         78.75 -0.30441833
## ENSG00000000971       5219.00       6687.50  0.35769358
## ENSG00000001036       2327.00       1785.75 -0.38194109
```

> Q7. What is the purpose of the arr.ind argument in the which() function call above? Why would we then take the first column of the output and need to call the unique() function?

The arr.ind = TRUE argument makes which() return the row and column positions of values equal to zero. Using unique() on the first column keeps each gene row only once, even if that gene contains zeros in multiple columns.

A common rule of thumb is to use a log2 fold-change cutoff of +2 for upregulated genes and -2 for downregulated genes.

> Q8. Using the up.ind vector above can you determine how many up regulated genes we have at the greater than 2 fc level?

The number of up-regulated genes is sum(up.ind) Number of "up" genes at +2 threshold

```
sum(meancounts$log2fc >= +2, na.rm=T)
```

```
## [1] 1910
```

1910 up regulated genes.

> Q9. Using the down.ind vector above can you determine how many down regulated genes we have at the greater than 2 fc level?

The number of down-regulated genes is sum(down.ind)

Number of "down" genes at -2 threshold

```
sum(meancounts$log2fc >= -2, na.rm=T)
```

```
## [1] 23046
```

23046 down regulated genes.

> Q10. Do you trust these results? Why or why not?

These results are based only on fold change, without normalization or statistical significance testing such as DESeq2 p-values.

DESeq2 analysis

Let's do this analysis properly and keep our inner stats nerd happy - i.e. are the differences we see between drug and no drug significant given the replicate experiments.

```
library(DESeq2)
```

```
dds <- DESeqDataSetFromMatrix(countData = counts,
                              colData = metadata,
                              design = ~dex)
```

```
## converting counts to integer mode
```

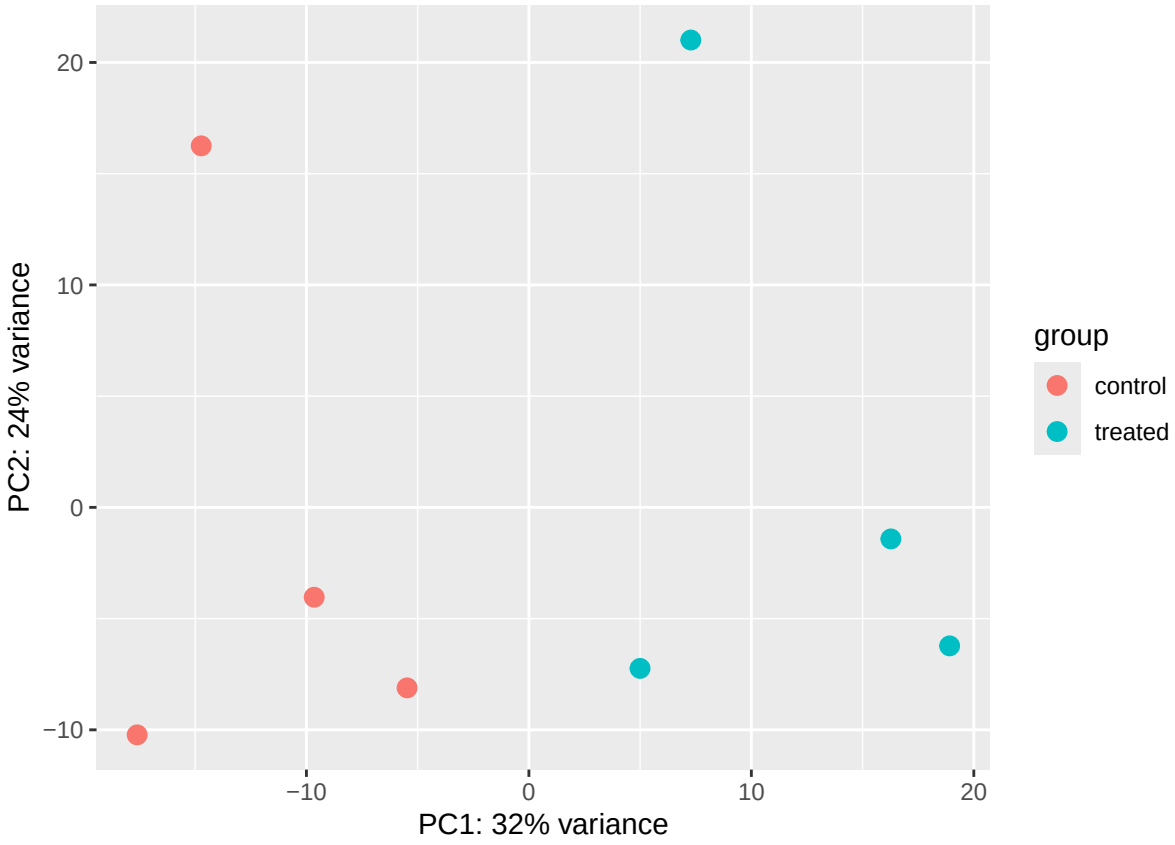
```
## Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in
## design formula are characters, converting to factors
```

Principal Component Analysis (PCA)

Before running DESeq analysis we can look how the count data samples are related to one another via our old friend Principal Component Analysis (PCA). We will follow the DESeq recommended procedure and associated functions for PCA. First calling vst() to apply a variance stabilizing transformation (read more about this in the expandable section below) and then plotPCA() to calculate our PCs and plot the results.

```
vsd <- vst(dds, blind = FALSE)
plotPCA(vsd, intgroup = c("dex"))
```

```
## using ntop=500 top features by variance
```




```
pcaData <- plotPCA(vsd, intgroup=c("dex"), returnData=TRUE)
```

```
## using ntop=500 top features by variance
```

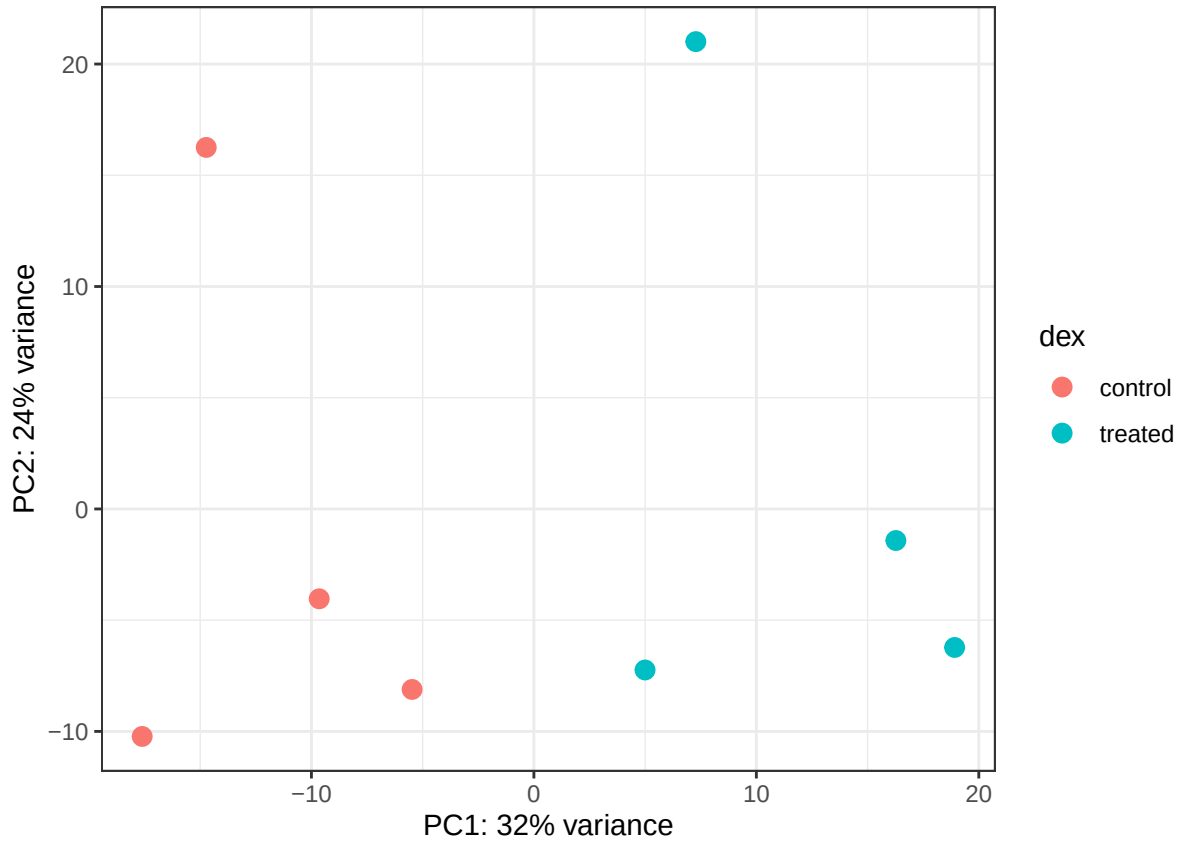
```
head(pcaData)
```

```
##                   PC1        PC2   group       name         id     dex celltype
## SRR1039508 -17.607922 -10.225252 control SRR1039508 SRR1039508 control   N61311
```

```
## SRR1039509   4.996738  -7.238117 treated SRR1039509 SRR1039509 treated   N61311
## SRR1039512  -5.474456  -8.113993 control SRR1039512 SRR1039512 control  N052611
## SRR1039513  18.912974  -6.226041 treated SRR1039513 SRR1039513 treated  N052611
## SRR1039516 -14.729173  16.252000 control SRR1039516 SRR1039516 control  N080611
## SRR1039517   7.279863  21.008034 treated SRR1039517 SRR1039517 treated  N080611
##                geo_id sizeFactor
## SRR1039508 GSM1275862  1.0193796
## SRR1039509 GSM1275863  0.9005653
## SRR1039512 GSM1275866  1.1784239
## SRR1039513 GSM1275867  0.6709854
## SRR1039516 GSM1275870  1.1731984
## SRR1039517 GSM1275871  1.3929361
```

```
# Calculate percent variance per PC for the plot axis labels
percentVar <- round(100 * attr(pcaData, "percentVar"))
```

```
ggplot(pcaData) +
  aes(x = PC1, y = PC2, color = dex) +
  geom_point(size =3) +
  xlab(paste0("PC1: ", percentVar[1], "% variance")) +
  ylab(paste0("PC2: ", percentVar[2], "% variance")) +
  coord_fixed() +
  theme_bw()
```



DESeq analysis

```
dds <- DESeq(dds)
```

```
## estimating size factors
## estimating dispersions
## gene-wise dispersion estimates
## mean-dispersion relationship
## final dispersion estimates
## fitting model and testing
```

```
res <- results(dds)
head(res)
```

```
## log2 fold change (MLE): dex treated vs control
## Wald test p-value: dex treated vs control
## DataFrame with 6 rows and 6 columns
##                   baseMean log2FoldChange     lfcSE      stat    pvalue
##                  <numeric>      <numeric> <numeric> <numeric> <numeric>
## ENSG00000000003 747.194195     -0.3507030  0.168246 -2.084470 0.0371175
## ENSG00000000005   0.000000             NA        NA        NA        NA
## ENSG00000000419 520.134160      0.2061078  0.101059  2.039475 0.0414026
## ENSG00000000457 322.664844      0.0245269  0.145145  0.168982 0.8658106
## ENSG00000000460  87.682625     -0.1471420  0.257007 -0.572521 0.5669691
## ENSG00000000938   0.319167     -1.7322890  3.493601 -0.495846 0.6200029
##                      padj
##                 <numeric>
## ENSG00000000003  0.163035
## ENSG00000000005        NA
## ENSG00000000419  0.176032
## ENSG00000000457  0.961694
## ENSG00000000460  0.815849
## ENSG00000000938        NA
```

```
summary(res)
```

```
##
## out of 25258 with nonzero total read count
## adjusted p-value < 0.1
## LFC > 0 (up)       : 1563, 6.2%
## LFC < 0 (down)     : 1188, 4.7%
## outliers [1]       : 142, 0.56%
## low counts [2]     : 9971, 39%
## (mean count < 10)
## [1] see 'cooksCutoff' argument of ?results
## [2] see 'independentFiltering' argument of ?results
```

```
res05 <- results(dds, alpha=0.05)
summary(res05)
```

```
##
## out of 25258 with nonzero total read count
## adjusted p-value < 0.05
## LFC > 0 (up)       : 1236, 4.9%
```

```
## LFC < 0 (down)     : 933, 3.7%
## outliers [1]       : 142, 0.56%
## low counts [2]     : 9033, 36%
## (mean count < 6)
## [1] see 'cooksCutoff' argument of ?results
## [2] see 'independentFiltering' argument of ?results
```

Adding Annotation Data

```
library("AnnotationDbi")
library("org.Hs.eg.db")
```

```
##
```

```
columns(org.Hs.eg.db)
```

```
##  [1] "ACCNUM"       "ALIAS"        "ENSEMBL"      "ENSEMBLPROT"  "ENSEMBLTRANS"
##  [6] "ENTREZID"     "ENZYME"       "EVIDENCE"     "EVIDENCEALL"  "GENENAME"
## [11] "GENETYPE"     "GO"           "GOALL"        "IPI"          "MAP"
## [16] "OMIM"         "ONTOLOGY"     "ONTOLOGYALL"  "PATH"         "PFAM"
## [21] "PMID"         "PROSITE"      "REFSEQ"       "SYMBOL"       "UCSCKG"
## [26] "UNIPROT"
```

```
res$symbol <- mapIds(org.Hs.eg.db,
                     keys=row.names(res), # Our genenames
                     keytype="ENSEMBL",        # The format of our genenames
                     column="SYMBOL",          # The new format we want to add
                     multiVals="first")
```

```
## 'select()' returned 1:many mapping between keys and columns
```

```
head(res)
```

```
## log2 fold change (MLE): dex treated vs control
## Wald test p-value: dex treated vs control
## DataFrame with 6 rows and 7 columns
##                   baseMean log2FoldChange     lfcSE      stat    pvalue
##                  <numeric>      <numeric> <numeric> <numeric> <numeric>
## ENSG00000000003 747.194195     -0.3507030  0.168246 -2.084470 0.0371175
## ENSG00000000005   0.000000             NA        NA        NA        NA
## ENSG00000000419 520.134160      0.2061078  0.101059  2.039475 0.0414026
## ENSG00000000457 322.664844      0.0245269  0.145145  0.168982 0.8658106
## ENSG00000000460  87.682625     -0.1471420  0.257007 -0.572521 0.5669691
## ENSG00000000938   0.319167     -1.7322890  3.493601 -0.495846 0.6200029
##                      padj      symbol
##                 <numeric> <character>
## ENSG00000000003  0.163035      TSPAN6
## ENSG00000000005        NA        TNMD
## ENSG00000000419  0.176032        DPM1
## ENSG00000000457  0.961694       SCYL3
## ENSG00000000460  0.815849       FIRRM
## ENSG00000000938        NA         FGR
```

Q11. Run the mapIds() function two more times to add the Entrez ID and UniProt accession and GENENAME as new columns called res$entrez, res$uniprot and res$genename.

```
res$entrez <- mapIds(org.Hs.eg.db,
                     keys=row.names(res),
                     column="ENTREZID",
                     keytype="ENSEMBL",
                     multiVals="first")
```

```
## 'select()' returned 1:many mapping between keys and columns
```

```
res$uniprot <- mapIds(org.Hs.eg.db,
                     keys=row.names(res),
                     column="UNIPROT",
                     keytype="ENSEMBL",
                     multiVals="first")
```

```
## 'select()' returned 1:many mapping between keys and columns
```

```
res$genename <- mapIds(org.Hs.eg.db,
                     keys=row.names(res),
                     column="GENENAME",
                     keytype="ENSEMBL",
                     multiVals="first")
```

```
## 'select()' returned 1:many mapping between keys and columns
```

```
head(res)
```

```
## log2 fold change (MLE): dex treated vs control
## Wald test p-value: dex treated vs control
## DataFrame with 6 rows and 10 columns
##                   baseMean log2FoldChange     lfcSE      stat    pvalue
##                  <numeric>      <numeric> <numeric> <numeric> <numeric>
## ENSG00000000003 747.194195     -0.3507030  0.168246 -2.084470 0.0371175
## ENSG00000000005   0.000000             NA        NA        NA        NA
## ENSG00000000419 520.134160      0.2061078  0.101059  2.039475 0.0414026
## ENSG00000000457 322.664844      0.0245269  0.145145  0.168982 0.8658106
## ENSG00000000460  87.682625     -0.1471420  0.257007 -0.572521 0.5669691
## ENSG00000000938   0.319167     -1.7322890  3.493601 -0.495846 0.6200029
##                      padj      symbol      entrez     uniprot
##                 <numeric> <character> <character> <character>
## ENSG00000000003  0.163035      TSPAN6        7105  A0A087WYV6
## ENSG00000000005        NA        TNMD       64102      Q9H2S6
## ENSG00000000419  0.176032        DPM1        8813      H0Y368
## ENSG00000000457  0.961694       SCYL3       57147      X6RHX1
## ENSG00000000460  0.815849       FIRRM       55732      A6NFP1
## ENSG00000000938        NA         FGR        2268      B7Z6W7
##                           genename
##                          <character>
## ENSG00000000003          tetraspanin 6
## ENSG00000000005            tenomodulin
## ENSG00000000419 dolichyl-phosphate m..
## ENSG00000000457 SCY1 like pseudokina..
## ENSG00000000460 FIGNL1 interacting r..
## ENSG00000000938 FGR proto-oncogene, ..
```

```
ord <- order( res$padj )
#View(res[ord,])
head(res[ord,])
```

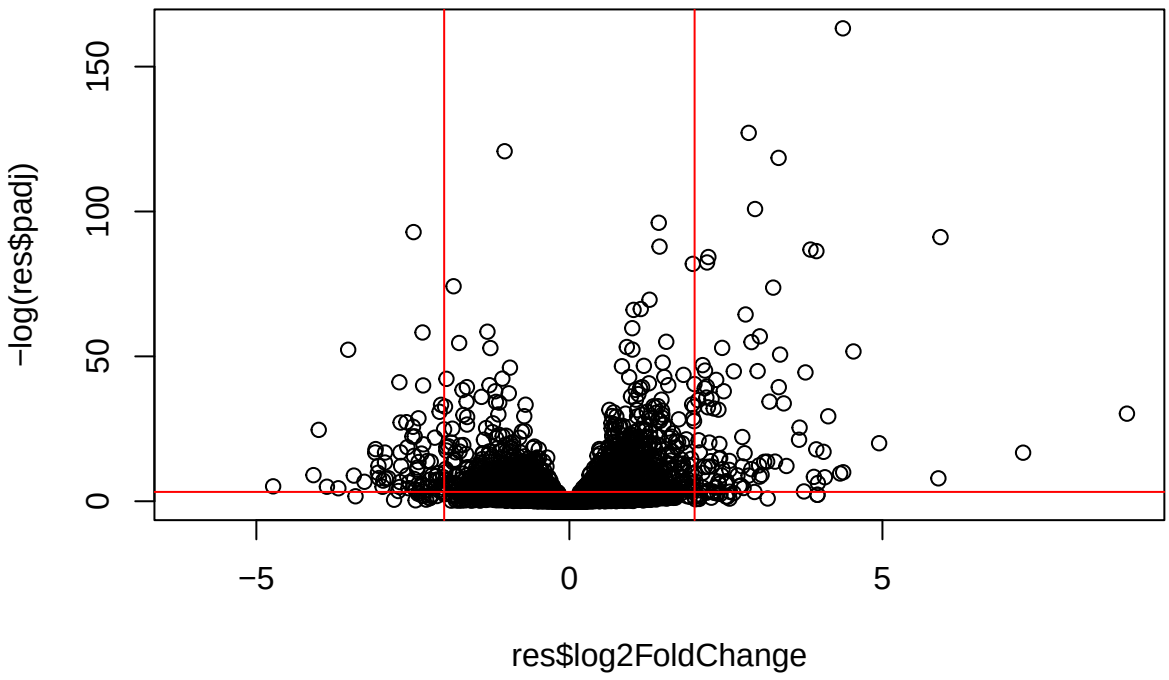
```
## log2 fold change (MLE): dex treated vs control
## Wald test p-value: dex treated vs control
## DataFrame with 6 rows and 10 columns
##                  baseMean log2FoldChange     lfcSE      stat      pvalue
##                 <numeric>      <numeric> <numeric> <numeric>   <numeric>
## ENSG00000152583   954.771        4.36836 0.2371268   18.4220 8.74490e-76
## ENSG00000179094   743.253        2.86389 0.1755693   16.3120 8.10784e-60
## ENSG00000116584  2277.913       -1.03470 0.0650984  -15.8944 6.92855e-57
## ENSG00000189221  2383.754        3.34154 0.2124058   15.7319 9.14433e-56
## ENSG00000120129  3440.704        2.96521 0.2036951   14.5571 5.26424e-48
## ENSG00000148175 13493.920        1.42717 0.1003890   14.2164 7.25128e-46
##                        padj      symbol      entrez     uniprot
##                   <numeric> <character> <character> <character>
## ENSG00000152583 1.32441e-71     SPARCL1        8404      B4E2Z0
## ENSG00000179094 6.13966e-56        PER1        5187      A2I2P6
## ENSG00000116584 3.49776e-53     ARHGEF2        9181  A0A8Q3SIN5
## ENSG00000189221 3.46227e-52        MAOA        4128      B4DF46
## ENSG00000120129 1.59454e-44       DUSP1        1843      B4DRR4
## ENSG00000148175 1.83034e-42        STOM        2040      F8VSL7
##                              genename
##                           <character>
## ENSG00000152583           SPARC like 1
## ENSG00000179094 period circadian reg..
## ENSG00000116584 Rho/Rac guanine nucl..
## ENSG00000189221    monoamine oxidase A
## ENSG00000120129 dual specificity pho..
## ENSG00000148175               stomatin
```

```
write.csv(res[ord,], "deseq_results.csv")
```

Data Visualization - Volcano Plot

Let's make a commonly produced visualization from this data, namely a so-called Volcano plot. These summary figures are frequently used to highlight the proportion of genes that are both significantly regulated and display a high fold change.

```
plot(res$log2FoldChange, -log(res$padj))
abline(v=c(-2,2), col="red")
abline(h=-log(0.04), col="red")
```




```
log(0.1)
```

```
## [1] -2.302585
```

```
log(0.000001)
```

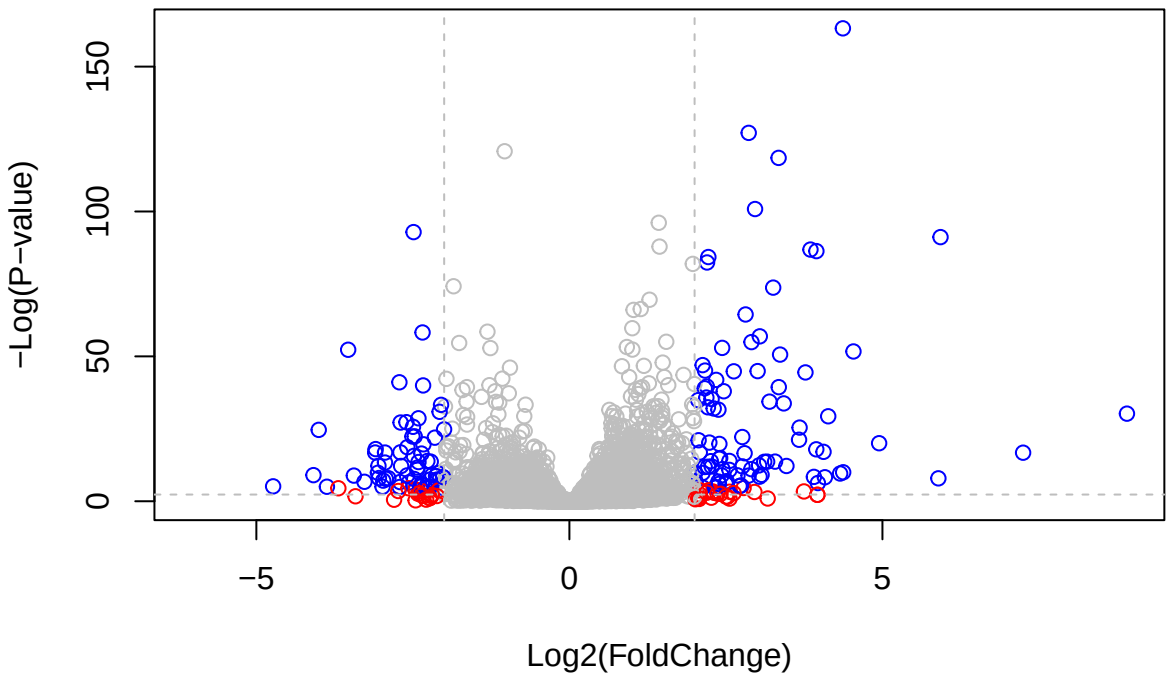
```
## [1] -13.81551
```

```
# Setup our custom point color vector
mycols <- rep("gray", nrow(res))
mycols[ abs(res$log2FoldChange) > 2 ]  <- "red"

inds <- (res$padj < 0.01) & (abs(res$log2FoldChange) > 2 )
mycols[ inds ] <- "blue"

# Volcano plot with custom colors
plot( res$log2FoldChange,  -log(res$padj),
 col=mycols, ylab="-Log(P-value)", xlab="Log2(FoldChange)" )

# Cut-off lines
abline(v=c(-2,2), col="gray", lty=2)
abline(h=-log(0.1), col="gray", lty=2)
```




```
library(EnhancedVolcano)
```

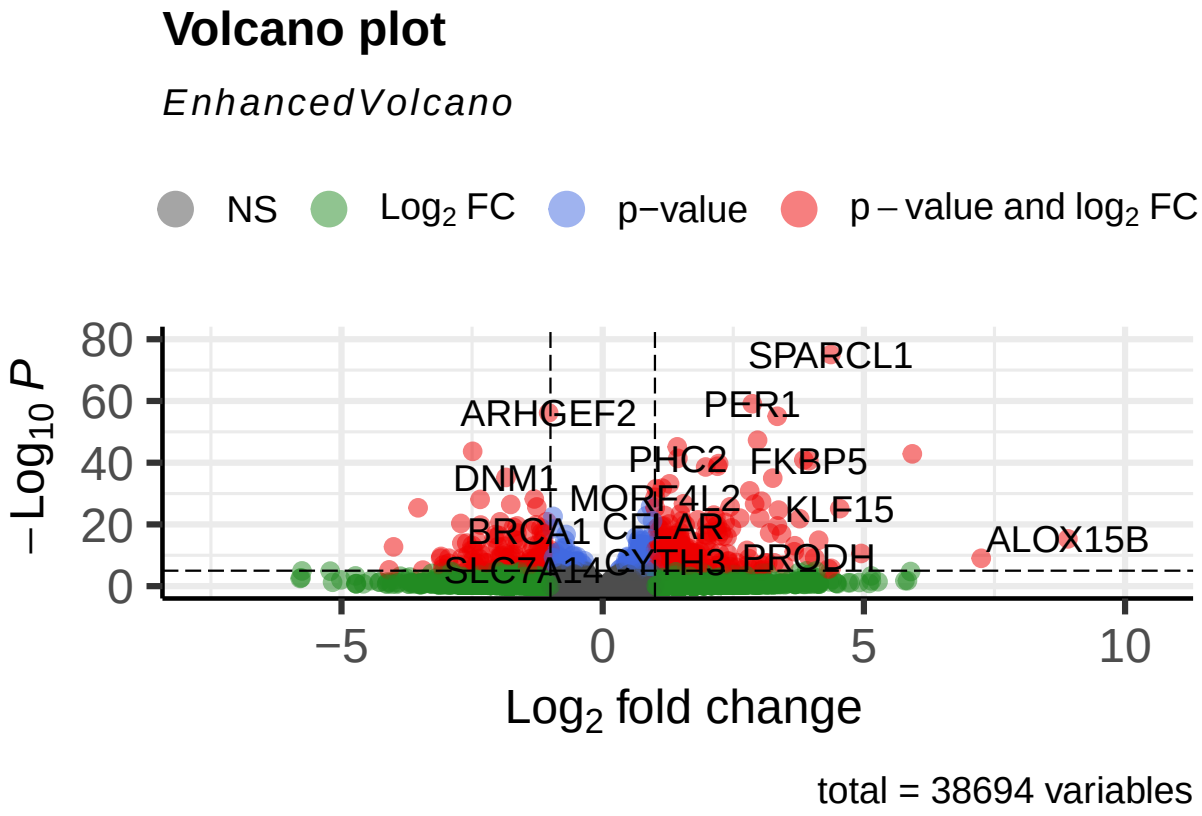
```
## Loading required package: ggrepel
```

```
x <- as.data.frame(res)

EnhancedVolcano(x,
    lab = x$symbol,
    x = 'log2FoldChange',
    y = 'pvalue')
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## i The deprecated feature was likely used in the EnhancedVolcano package.
##   Please report the issue to the authors.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
## Warning: The `size` argument of `element_line()` is deprecated as of ggplot2 3.4.0.
## i Please use the `linewidth` argument instead.
## i The deprecated feature was likely used in the EnhancedVolcano package.
##   Please report the issue to the authors.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```
write.csv(res, file="my_results.csv")
```

Pathway analysis

```
library(gage)
library(gageData)
library(pathview)
```

```
foldchanges <- res$log2FoldChange
names(foldchanges) <- res$entrez
head(foldchanges)
```

```
##        7105       64102        8813       57147       55732        2268
## -0.35070302          NA  0.20610777  0.02452695 -0.14714205 -1.73228897
```

```
data(kegg.sets.hs)

keggres = gage(foldchanges, gsets=kegg.sets.hs)
```

In our results object we have:

```
attributes(keggres)
```

```
## $names
## [1] "greater" "less"    "stats"
```

```
head(keggres$less, 5)
```

```
##                                                       p.geomean stat.mean
## hsa05332 Graft-versus-host disease                   0.0004250461 -3.473346
## hsa04940 Type I diabetes mellitus                    0.0017820293 -3.002352
## hsa05310 Asthma                                      0.0020045888 -3.009050
## hsa04672 Intestinal immune network for IgA production 0.0060434515 -2.560547
## hsa05330 Allograft rejection                         0.0073678825 -2.501419
##                                                             p.val      q.val
## hsa05332 Graft-versus-host disease                   0.0004250461 0.09053483
## hsa04940 Type I diabetes mellitus                    0.0017820293 0.14232581
## hsa05310 Asthma                                      0.0020045888 0.14232581
## hsa04672 Intestinal immune network for IgA production 0.0060434515 0.31387180
## hsa05330 Allograft rejection                         0.0073678825 0.31387180
##                                                      set.size         exp1
## hsa05332 Graft-versus-host disease                         40 0.0004250461
## hsa04940 Type I diabetes mellitus                          42 0.0017820293
## hsa05310 Asthma                                            29 0.0020045888
## hsa04672 Intestinal immune network for IgA production      47 0.0060434515
## hsa05330 Allograft rejection                               36 0.0073678825
```

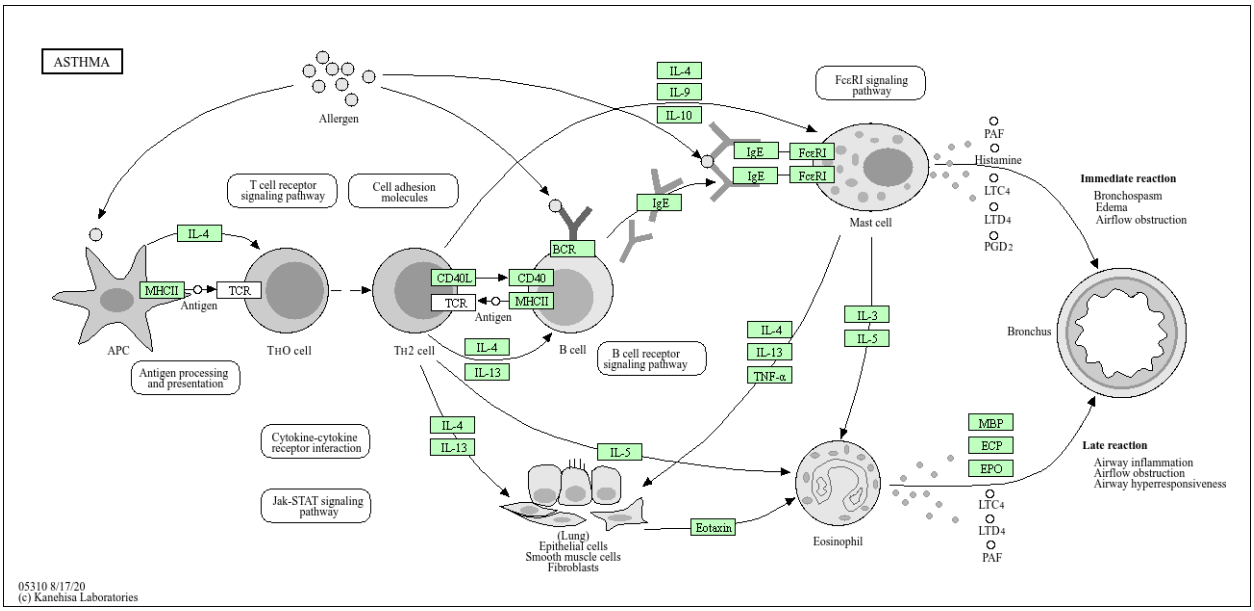
```
pathview(pathway.id = "hsa05310", gene.data = foldchanges)
```

```
## 'select()' returned 1:1 mapping between keys and columns
```

```
## Info: Working in directory C:/Users/chenr/OneDrive/Desktop/class 013
```

```
## Info: Writing image file hsa05310.pathview.png
```

Add this pathway figure to our lab report




```
write.csv(res, file = "myresults_annotated.csv")
```